

CS252M
OPERATING SYSTEMS

PROJECT REPORT

USB DEVICE GUARD KERNEL MODULE

Submitted by:

Shubhang S. Galagali - **23IMT049**

Suhas Gowda M. - **23IMT053**

Vijeeth J. Poojary - **23IEE263**

Submitted to:

Dr. Radhika B. S.

Assistant Professor,

Department of Computer Science and Engineering,

NITK Surathkal.



National Institute of Technology Karnataka [NITK], Surathkal.

NH 66, Srinivasnagar, Surathkal, Mangalore, Karnataka- 575025.

Abstract:

The USB Device Guard Kernel Module is a **Linux kernel module** designed to provide a robust, low-level mechanism for **monitoring and restricting USB device connections**. This project focuses on enhancing system security by implementing access control based on specific device identifiers, namely the **Vendor ID (VID)**, **Product ID (PID)**, and **serial numbers**. Unauthorised devices are blocked during the connection phase, preventing potential security breaches or malware injection. The module features **dynamic rule configuration** managed through the `sysfs` interface and supports static rule loading from a dedicated configuration file (`/etc/usbguard.rules`) upon initialisation. Detailed connection, disconnection, and access decisions are logged to the kernel ring buffer for auditing purposes.

1. Motivation:

USB peripherals are ubiquitous and inherently trusted by host systems after they are enumerated. This trust exposes endpoints to hardware implants, rogue mass-storage devices, HID spoofers (e.g., keystroke injection), and unauthorized data exfiltration. Traditional user-space mitigations (udev rules, policy daemons) are effective but react after enumeration; a kernel-resident control path allows earlier enforcement and simpler provenance for audit.

2. Introduction:

This kernel module, fixed for compatibility with **modern Linux kernel versions (specifically 6.x and later, tested on 6.12.11)**, operates by hooking into the USB subsystem to intercept device connection events. By enforcing a mandatory access control policy at the kernel level, the module ensures that only explicitly permitted devices (identified by their VID/PID or non-blocked serial number) are allowed to function. The module leverages `sysfs` to enable root users to manage access rules dynamically and block serial lists without requiring a module reload.

3. Problem Statement:

Traditional operating system security mechanisms often fail to adequately restrict malicious or unauthorised USB device connections immediately upon plug-in. This delay or lack of strict kernel-level control exposes systems to risks such as:

1. **Unauthorised Data Access:** Uncontrolled access to peripheral ports enables the connection of untrusted devices, potentially compromising sensitive data.
2. **Physical Attacks:** In scenarios where a device is physically compromised or stolen while unlocked, unauthorised devices (such as USB wipers) can be used to maintain system access, potentially bypassing locked screens and keeping data decrypted in memory.
3. **Lack of Granular Control:** System administrators require a mechanism to whitelist or blacklist devices based on unique hardware attributes (like VID, PID, and serial number) rather than relying solely on high-level security frameworks.

The core problem is the need for **dead-accurate, immediate security enforcement** against USB devices handled *before* the operating system assigns them resources.

4. Methodology:

The USB Device Guard project utilises a **Linux USB driver/kernel module architecture**. The implementation details are rooted in C programming tailored for the kernel environment.

A. Rule Management:

1. **Static Loading:** Upon module initialisation (`usbguard_init`), rules are loaded from the configuration file, `/etc/usbguard.rules`. This process involves parsing the file line by line, **trimming whitespace**, ignoring comments (denoted by `#`), and storing valid **VID/PID pairs** in a dynamic array (`rules`). The current maximum capacity for rules is defined as 128 (`MAX_RULES`) in the source code.
2. **Dynamic Configuration:** Rule management and blocked serial lists are exposed via `sysfs`.

- `/sys/kernel/usbguard/rules`: Allows users to view rules (`rules_show`) or add new VID/PID rules dynamically (`rules_store`).
- `/sys/kernel/usbguard/blocked_serials`: Allows users to view or add serial numbers to a list of blocked identifiers.

B. Access Control Flow (Probe Function)

The primary enforcement mechanism resides in the `usbguard_probe` function, which is executed when a new USB device connects. The function performs checks sequentially:

1. **Logging:** Records the device's attached VID and PID.
2. **VID/PID Matching:** Checks if the device's VID/PID matches any entry in the allowed `rules` array using `match_rules()`. If no match is found, the device is **rejected immediately with the -EACCES error**.
3. **Interface Class Check:** Performs a basic check on the device interface classes (although defined as a stub, `check_interface_classes()` is designed to monitor classes such as USB mass storage). Rejection occurs if the class is disallowed.
4. **Serial Blocking Check:** If the device has a serial number, it is retrieved. The function `serial_blocked()` checks if this serial number exists in the dynamically managed block list. If blocked, the device is **rejected**.
5. **Acceptance:** If all checks pass, the device is accepted, and an information log entry is created.

Installation Method:

Installation is automated via the `install.sh` script. This script:

1. Ensures root privileges are maintained.
2. Builds the module using the `Makefile`.
3. Copies the compiled module (`usbguard.ko`) to the kernel directory (`/lib/modules/$(uname -r)/extra`).
4. Copies the default rule file to `/etc/usbguard.rules`.
5. Refreshes module dependencies (`depmod`).

6. Loads the module into the kernel (using `modprobe usbguard`) and ensures it is loaded at boot.

File structure:

- **usbguard.c**: Main source file containing the kernel module implementation.
- **Makefile**: Build script for compiling and managing the kernel module.
- **usbguard.rules**: Default rule file with sample configurations.
- **install.sh**: Installation script to set up the environment and copy necessary files.
- **README.md**: Documentation for the project.

5. Illustration of the working of LKM:

- **Loading of files**

```
bash: /opt/openfoam2212/etc/bashrc: No such file or directory
bash: /opt/openfoam2212/etc/bashrc: No such file or directory
No completions for /home/shubhang/OpenFOAM/openfoam/platforms/linux64GccDPInt32Opt/bin
[ignore if OpenFOAM is not yet compiled]
No completions for /home/shubhang/OpenFOAM/openfoam/platforms/linux64GccDPInt32Opt/bin
[ignore if OpenFOAM is not yet compiled]
No completions for /home/shubhang/OpenFOAM/openfoam/platforms/linux64GccDPInt32Opt/bin
[ignore if OpenFOAM is not yet compiled]
shubhang@Galagali-IdeaPad-3:~$ ls
Desktop  log.lammps  Open      Pictures  Templates
Documents MicroSim    OpenFOAM  Public    triangular_cylinder
Downloads Music       OS-CS     snap      Videos
shubhang@Galagali-IdeaPad-3:~$ cd OS-CS/
shubhang@Galagali-IdeaPad-3:~/OS-CS$ mkdir ~/OS_Project_Demo
shubhang@Galagali-IdeaPad-3:~/OS-CS$ cd ~/OS_Project_Demo
shubhang@Galagali-IdeaPad-3:~/OS_Project_Demo$ git clone https://github.com/galavashubhang/OS-Project.git
Cloning into 'OS-Project'...
remote: Enumerating objects: 82, done.
remote: Counting objects: 100% (82/82), done.
remote: Compressing objects: 100% (42/42), done.
remote: Total 82 (delta 39), reused 82 (delta 39), pack-reused 0 (from 0)
Receiving objects: 100% (82/82), 49.42 KiB | 1.18 MiB/s, done.
Resolving deltas: 100% (39/39), done.
shubhang@Galagali-IdeaPad-3:~/OS_Project_Demo$ ls
OS-Project
shubhang@Galagali-IdeaPad-3:~/OS_Project_Demo$ cd OS-Project/
shubhang@Galagali-IdeaPad-3:~/OS_Project_Demo/OS-Project$ ls
install.sh  LICENSE  Makefile  README.md  usbguard.c  usbguard.rules
shubhang@Galagali-IdeaPad-3:~/OS_Project_Demo/OS-Project$ make
make -C /lib/modules/6.8.0-45-generic/build M=/home/shubhang/OS_Project_Demo/OS-Project modules
make[1]: Entering directory '/usr/src/linux-headers-6.8.0-45-generic'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-12 (Ubuntu 12.3.0-1ubuntu1-22.04) 12.3.0
You are using:          gcc-12 (Ubuntu 12.3.0-1ubuntu1-22.04.2) 12.3.0
CC [M] /home/shubhang/OS_Project_Demo/OS-Project/usbguard.o
MODPOST /home/shubhang/OS_Project_Demo/OS-Project/Module.symvers
CC [M] /home/shubhang/OS_Project_Demo/OS-Project/usbguard.mod.o
LD [M] /home/shubhang/OS_Project_Demo/OS-Project/usbguard.ko
BTF [M] /home/shubhang/OS_Project_Demo/OS-Project/usbguard.ko
Skipping BTF generation for /home/shubhang/OS_Project_Demo/OS-Project/usbguard.ko due to unavailability of vmlinux
make[1]: Leaving directory '/usr/src/linux-headers-6.8.0-45-generic'
shubhang@Galagali-IdeaPad-3:~/OS_Project_Demo/OS-Project$ sudo insmod usbguard.ko
[sudo] password for shubhang: 
```

• Verification of the modules existing

```

bash: /opt/openfoam2212/etc/bashrc: No such file or directory
bash: /opt/openfoam2212/etc/bashrc: No such file or directory
No completions for /home/shubhang/OpenFOAM/openfoam/platforms/linux64GccDPInt32Opt/bin
[ignore if OpenFOAM is not yet compiled]
No completions for /home/shubhang/OpenFOAM/openfoam/platforms/linux64GccDPInt32Opt/bin
[ignore if OpenFOAM is not yet compiled]
shubhang@Galagali-IdeaPad-3: ~$ sudo lsmod usbguard.ko
[sudo] password for shubhang:
lsmod: ERROR: could not load module usbguard.ko: No such file or directory
shubhang@Galagali-IdeaPad-3: ~$ cd ~/OS-CS/gh-proj/USB-Device-Guard-Driver
ls -l
total 620
-rwxrwxr-x 1 shubhang shubhang 942 Nov 4 22:39 install.sh
-rw-rw-r-- 1 shubhang shubhang 18892 Nov 4 22:39 LICENSE
-rw-rw-r-- 1 shubhang shubhang 297 Nov 4 22:39 Makefile
-rw-rw-r-- 1 shubhang shubhang 64 Nov 4 22:49 modules.order
-rw-rw-r-- 1 shubhang shubhang 0 Nov 4 22:49 Module.symvers
-rw-rw-r-- 1 shubhang shubhang 4419 Nov 4 22:39 README.md
-rw-rw-r-- 1 shubhang shubhang 10277 Nov 4 22:49 usbguard.c
-rw-rw-r-- 1 shubhang shubhang 383272 Nov 4 22:49 usbguard.ko
-rw-rw-r-- 1 shubhang shubhang 64 Nov 4 22:49 usbguard.mod
-rw-rw-r-- 1 shubhang shubhang 1836 Nov 4 22:49 usbguard.mod.c
-rw-rw-r-- 1 shubhang shubhang 152256 Nov 4 22:49 usbguard.mod.o
-rw-rw-r-- 1 shubhang shubhang 232680 Nov 4 22:49 usbguard.o
-rw-rw-r-- 1 shubhang shubhang 603 Nov 4 22:39 usbguard.rules
shubhang@Galagali-IdeaPad-3: ~/OS-CS/gh-proj/USB-Device-Guard-Driver$ sudo lsmod .usbguard.ko
shubhang@Galagali-IdeaPad-3: ~/OS-CS/gh-proj/USB-Device-Guard-Driver$ cd ~/OS-CS/gh-proj/USB-Device-Guard-Driver
ls -l
total 620
-rwxrwxr-x 1 shubhang shubhang 942 Nov 4 22:39 install.sh
-rw-rw-r-- 1 shubhang shubhang 18892 Nov 4 22:39 LICENSE
-rw-rw-r-- 1 shubhang shubhang 297 Nov 4 22:39 Makefile
-rw-rw-r-- 1 shubhang shubhang 64 Nov 4 22:49 modules.order
-rw-rw-r-- 1 shubhang shubhang 0 Nov 4 22:49 Module.symvers
-rw-rw-r-- 1 shubhang shubhang 4419 Nov 4 22:39 README.md
-rw-rw-r-- 1 shubhang shubhang 10277 Nov 4 22:49 usbguard.c
-rw-rw-r-- 1 shubhang shubhang 383272 Nov 4 22:49 usbguard.ko
-rw-rw-r-- 1 shubhang shubhang 64 Nov 4 22:49 usbguard.mod
-rw-rw-r-- 1 shubhang shubhang 1836 Nov 4 22:49 usbguard.mod.c
-rw-rw-r-- 1 shubhang shubhang 152256 Nov 4 22:49 usbguard.mod.o
-rw-rw-r-- 1 shubhang shubhang 232680 Nov 4 22:49 usbguard.o
-rw-rw-r-- 1 shubhang shubhang 603 Nov 4 22:39 usbguard.rules
shubhang@Galagali-IdeaPad-3: ~/OS-CS/gh-proj/USB-Device-Guard-Driver$ sudo lsmod .usbguard.ko
lsmod: ERROR: could not insert module /home/shubhang/OS-CS/gh-proj/USB-Device-Guard-Driver/usbguard.ko
lsmod: ERROR: could not insert module /home/shubhang/OS-CS/gh-proj/USB-Device-Guard-Driver/usbguard.ko: File exists
shubhang@Galagali-IdeaPad-3: ~/OS-CS/gh-proj/USB-Device-Guard-Driver$ lsmod | grep usbguard
usbguard 16384 0
shubhang@Galagali-IdeaPad-3: ~/OS-CS/gh-proj/USB-Device-Guard-Driver$ ls /sys/usbguard
ls: cannot access '/sys/usbguard': No such file or directory
shubhang@Galagali-IdeaPad-3: ~/OS-CS/gh-proj/USB-Device-Guard-Driver$ sudo lsmod /home/shubhang/OS-CS/gh-proj/USB-Device-Guard-Driver/usbguard.ko
lsmod: ERROR: could not insert module /home/shubhang/OS-CS/gh-proj/USB-Device-Guard-Driver/usbguard.ko: File exists

```

• Restricting a particular device with a unique PID and VID

```

[ 1734.473326] usbguard: module unloaded
[ 1832.544577] usbcore: registered new interface driver usbguard_demo
[ 1832.544584] usbguard: module loaded
[ 1963.420964] usbcore: deregistering interface driver usbguard_demo
[ 1963.421018] usbguard: module unloaded
[ 2442.398974] usbcore: registered new interface driver usbguard_demo
[ 2442.398980] usbguard: module loaded
[ 2772.694511] usbcore: deregistering interface driver usbguard_demo
[ 2772.694558] usbguard: module unloaded
[ 2781.395631] usbcore: registered new interface driver usbguard_demo
[ 2781.395635] usbguard: module loaded
[ 3259.673287] usbcore: deregistering interface driver usbguard_demo
[ 3259.673317] usbguard: module unloaded
[ 3271.580641] usbguard: device VID=0951 PID=1643 attached
[ 3271.580653] usbguard: VID/PID not allowed, rejecting device
[ 3271.580658] usbguard_demo: probe of 1-1:1.0 failed with error -13
[ 3271.580707] usbcore: registered new interface driver usbguard_demo
[ 3271.580709] usbguard: module loaded
shubhang@Galagali-IdeaPad-3: ~/OS_Project_Demo/OS-Project$ sudo rmmod usbguard_demo
rmmod: ERROR: Module usbguard_demo is not currently loaded
shubhang@Galagali-IdeaPad-3: ~/OS_Project_Demo/OS-Project$ sudo rmmod usbguard
shubhang@Galagali-IdeaPad-3: ~/OS_Project_Demo/OS-Project$ lsmod | grep -i usbguard
shubhang@Galagali-IdeaPad-3: ~/OS_Project_Demo/OS-Project$ sudo insmod usbguard.ko
shubhang@Galagali-IdeaPad-3: ~/OS_Project_Demo/OS-Project$ lsusb -t
/: Bus 02.Port 1: Dev 1, Class=root_hub, Driver=xhci_hcd/6p, 10000M
/: Bus 01.Port 1: Dev 1, Class=root_hub, Driver=xhci_hcd/12p, 480M
    |__ Port 1: Dev 18, If 0, Class=Mass Storage, Driver=usb-storage, 480M
    |__ Port 2: Dev 3, If 0, Class=Human Interface Device, Driver=usbhid, 1.5M
    |__ Port 2: Dev 3, If 1, Class=Human Interface Device, Driver=usbhid, 1.5M
    |__ Port 5: Dev 4, If 1, Class=Video, Driver=uvccvideo, 480M
    |__ Port 5: Dev 4, If 0, Class=Video, Driver=uvccvideo, 480M
    |__ Port 9: Dev 5, If 0, Class=Human Interface Device, Driver=usbhid, 1.5M
    |__ Port 10: Dev 6, If 0, Class=Wireless, Driver=btusb, 12M
    |__ Port 10: Dev 6, If 1, Class=Wireless, Driver=btusb, 12M

```


- Loading and verifying the details of the device inserted into the system:

```
ce= 0.00
[ 7428.893560] usb 1-9: New USB device strings: Mfr=1, Product=2, SerialNumber=3
[ 7428.893561] usb 1-9: Product: DataTraveler G3
[ 7428.893563] usb 1-9: Manufacturer: KINGSTON
[ 7428.893564] usb 1-9: SerialNumber: 001CC07CEDAEACA1691804D2
[ 7428.895473] usb-storage 1-9:1.0: USB Mass Storage device detected
[ 7428.896086] scsi host1: usb-storage 1-9:1.0
[ 7430.735033] scsi 1:0:0:0: Direct-Access      KINGSTON DataTraveler G3  1.00 PQ: 0
ANSI: 4
[ 7430.735555] sd 1:0:0:0: Attached scsi generic sg1 type 0
[ 7430.736780] sd 1:0:0:0: [sdb] 15204352 512-byte logical blocks: (7.78 GB/7.25 GiB
)
[ 7430.738797] sd 1:0:0:0: [sdb] Write Protect is off
[ 7430.738801] sd 1:0:0:0: [sdb] Mode Sense: 2f 00 00 00
[ 7430.740793] sd 1:0:0:0: [sdb] Write cache: disabled, read cache: enabled, doesn't
support DPO or FUA
[ 7430.747252] sdb: sdb1
[ 7430.747453] sd 1:0:0:0: [sdb] Attached SCSI removable disk
[ 7431.106610] FAT-fs (sdb1): Volume was not properly unmounted. Some data may be co
rrupt. Please run fsck.
[ 7482.312897] usbcore: deregistering interface driver usbguard_demo
[ 7482.312930] usbguard: Module Unloaded.
[ 7482.339553] usbcore: registered new interface driver usbguard_demo
[ 7482.339559] usbguard: Kernel Module Registered and Active.
```

6. Conclusion:

The USB Device Guard Kernel Module successfully achieves its objective of providing a robust, **kernel-resident access control layer** for USB device connections on Linux systems. By implementing checks based on VID/PID whitelisting and serial number blacklisting, the project demonstrates effective security enforcement, blocking unauthorized devices immediately upon connection. The utilization of `sysfs` provides necessary flexibility for dynamic rule adjustments, moving beyond static configuration files and enabling **real-time security policy updates**.

7. Future work:

The project currently acknowledges only finite maximum rule capacity and the manual nature of updating rules in `/etc/usbguard.rules`. Future work focuses on automating rule persistence and expanding the complexity of device class checks beyond the current stub implementation.

8. References:

1. [USBGuard](#) - Documentation Page.
 2. [USBGuard: authorization for USB \[LWN.net\]](#)
 3.  USB Guard : Protecting Your System on Linux
-

GitHub - Project Repository:

<https://github.com/galava-shubhang/CS252M-OS-Project-USBGuard-Demo>